

RTC Platform

Recommended Modifications & Startup Strategy

A practical product and roadmap document based on the uploaded RTC Platform specification and the review discussed in this conversation.

Current foundation	Recommended direction
RTC infrastructure: signaling, WebRTC, P2P/SFU, TURN, Redis, PostgreSQL and SDK.	Realtime Communication + AI platform with developer tooling, communication intelligence and focused vertical workflows.

Key recommendation

Keep the existing RTC engine, but do not launch it as simply another generic RTC infrastructure provider. Move the commercial value one layer upward: developer experience, observability, AI communication intelligence and one focused business workflow.

1. Executive Summary

The uploaded document describes an embeddable RTC platform with chat, 1:1 voice, group voice, SDK integration, signaling, mediasoup SFU, Redis-backed scaling, PostgreSQL persistence and coturn/TURN support. The technical foundation is strong enough to continue developing.

The main weakness is strategic positioning. A generic RTC platform places the product in a difficult category against established providers. The recommendation is therefore to preserve the infrastructure work while building a differentiated product layer above it.

Recommended positioning: “Realtime communication infrastructure for AI-powered applications.”

2. What the Current Product Already Provides

Area	Current direction
Authentication	App registration, App ID/secret and JWT-based room authentication.
Signaling	Redis-backed WebSocket signaling.
RTC	WebRTC with P2P and SFU approaches.
Media	mediasoup-based SFU and coturn/TURN support.
Chat	Room messaging through the realtime signaling layer.
SDK	Developer SDK with room, messaging and voice-related methods.
Backend	API layer with PostgreSQL and Redis.
Deployment	Docker-oriented infrastructure with production scaling in the roadmap.

3. Main Strategic Gap

Current positioning: “RTC infrastructure for applications.”

Customers usually do not want to buy signaling, SFUs or TURN servers as an end goal. They want an outcome such as “let my customer call my agent” or “add realtime video and AI transcription to my application.”

Recommended product statement: “Add realtime voice, video, chat, transcription and AI agents to your application through one developer-friendly SDK—without building WebRTC, media servers, TURN infrastructure or realtime AI pipelines yourself.”

4. Recommended Product Modifications

4.1 Developer Platform

- Add self-service dashboard: projects, API keys, environments, logs and usage.
- Make first successful call possible within minutes.
- Hide mediasoup/TURN/ICE complexity behind simple SDK APIs.
- Add REST APIs and signed webhooks.

4.2 Video

- Add 1:1 and group video if remaining a general RTC platform.
- Prioritize camera controls, mute/unmute, screen sharing and reconnect handling.
- Do not overbuild advanced video features before customer validation.

4.3 Production Chat

- Message history and pagination.

- Read/delivery status, typing and presence.
- Attachments, replies, reactions and moderation.
- Push notifications.

4.4 Recording + AI

- Session recording with access control and retention.
- Speech-to-text/transcription.
- AI call summary and action items.
- Optional sentiment/topic extraction.
- Events when recording, transcript or summary is ready.

4.5 Webhooks

- call.started, ringing, connected, ended and failed.
- user.joined, user.left and message.sent.
- recording.ready, transcript.ready and summary.ready.
- Webhook signing, retries and delivery logs.

4.6 Usage + Billing

- Voice/video participant minutes, recording storage, messages and AI usage.
- Quotas, plan limits, invoices and usage dashboards.
- Keep billing architecture simple until usage is validated.

4.7 RTC Observability

- Latency, jitter, packet loss, bitrate and reconnects.
- ICE failures and TURN usage.
- Call quality score and failure reason.
- Per-project and per-room diagnostics.

4.8 Security

- Key rotation, tenant isolation and role-based access.
- Audit logs and signed webhooks.
- Recording access controls and retention policies.
- Rate limiting and abuse prevention.

5. Recommended Architecture Direction

Layer	Responsibility
Client SDKs	Web first; Flutter/mobile wrappers as demand appears.
API Gateway	Projects, credentials, tokens, configuration and webhooks.
Realtime Layer	WebSocket signaling, room state and presence.
Media Layer	P2P/SFU, mediasoup and TURN/coturn.
Communication Layer	Chat, call state, recording and notifications.
Intelligence Layer	Transcription, summaries, actions and AI agents.
Observability	Call quality, logs, metrics and failure diagnostics.
Billing	Usage metering, quotas, plans and invoices.
Dashboard	Project setup, keys, usage, rooms, logs, analytics and billing.

6. The Product Workflow to Aim For

Developer: Create project → get credentials → install SDK → first call → quality dashboard → webhook → recording/transcript/AI output.

Example support workflow: Customer clicks Call → agent receives call → RTC connects → recording/transcription → AI summary → action items → webhook creates or updates a ticket.

This makes RTC the engine underneath a valuable business workflow rather than the entire product proposition.

7. Important Gaps and Priority

Gap	Why it matters	Priority
No killer use case	Generic RTC makes differentiation and sales difficult.	Critical
No developer dashboard	Customers need self-service setup and visibility.	High
No video yet	Limits competitiveness if positioned as general RTC.	High
Basic chat	Needs production communication features.	Medium
No recording/transcription	Misses communication intelligence opportunities.	High
Limited AI layer	AI can become a major differentiator.	High
Usage/billing	Required for a commercial infrastructure business.	High
Observability	RTC failures are immediately visible to users.	High
Webhooks	Customers need to connect RTC events to their systems.	High
Mobile/push workflow	Incoming calls must work when apps are closed.	High
Global scaling	Important after demand; expensive to overbuild early.	Later

8. Differentiation Strategy

- **Developer experience:** first production-quality call in minutes.
- **AI-native communication:** transcription, summaries, actions and agents around realtime sessions.
- **Explainable observability:** make call quality and failures understandable.
- **Vertical workflows:** pre-built patterns for a specific customer segment.
- **India-first opportunity:** where useful, optimize onboarding, pricing, payments and support for Indian startups/SMBs.

The strongest direction is not “cheaper than Agora.” It is a combination of easy integration + realtime infrastructure + communication intelligence + a focused workflow.

9. Recommended Roadmap

Phase	Build	Goal
1	Stabilize current SDK, auth, rooms, chat, voice, P2P/SFU and TURN.	Reliable technical foundation.
2	Dashboard, projects, API keys, logs, webhooks and basic metering.	Become a usable SaaS.
3	1:1/group video, screen share, push and robust call states.	Close core RTC gaps.
4	Recording, transcription, AI summaries and action extraction.	Create intelligence layer.
5	One vertical workflow + AI-agent capability.	Create differentiation.
6	Multi-region, Kubernetes and enterprise controls.	Scale after validated demand.

10. Recommended MVP

MVP A — Developer-ready RTC

- Web SDK; authentication; 1:1/group voice; basic chat; P2P/SFU; TURN; dashboard; webhooks; call logs.

MVP B — Communication Intelligence

- Recording; transcription; AI summary; action items; searchable call history.

MVP C — One Vertical

- Choose one segment such as customer support, telehealth or education after customer interviews, then provide pre-configured workflows on top of the RTC engine.

11. Example Vertical: Customer Support

Customer event	Platform action
Clicks "Call Support"	Creates session and alerts/routes available agent.
Agent answers	RTC establishes connection.
Conversation	Captures quality metrics and optionally records.
Call ends	Stores call metadata.
Processing finishes	Creates transcript and AI summary.
Action identified	Webhook updates CRM/ticketing system.

The same infrastructure can later support other verticals. The first commercial workflow should be narrow enough to sell clearly and validate quickly.

12. Business Model Direction

Revenue layer	Possible model
RTC usage	Voice/video participant-minute pricing or tiered plans.
Recording	Storage and processing pricing.
AI	Transcription/AI usage pricing or bundled quotas.
Enterprise	Higher limits, security, dedicated infrastructure and support.
Vertical modules	Premium subscription for workflow-specific features.

Start with software revenue. Avoid adding marketplace or financing complexity until there is proven product usage.

13. What Not to Build Yet

- Do not immediately build a huge multi-region Kubernetes system before customer demand requires it.
- Do not build dozens of SDKs simultaneously.
- Do not overbuild chat before the core RTC workflow is reliable.
- Do not compete mainly on price.
- Do not market to every industry at launch.
- Do not add AI as a superficial chatbot; connect AI to realtime sessions and business actions.

14. Final Startup Assessment

Dimension	Assessment
Technical foundation	Very strong starting point.
Generic RTC differentiation	Weak without a specific advantage.
Competition	Very high in generic RTC infrastructure.
AI opportunity	High.
Developer SaaS opportunity	High if onboarding and reliability are excellent.
Vertical SaaS opportunity	Potentially very high with the right niche.
Recommended strategy	Keep RTC engine; move product positioning and value upward.

Verdict: Continue developing the project, but do not launch it simply as a generic RTC clone. The better startup path is to turn the existing technical foundation into a developer-friendly realtime

communication + AI platform and validate one specific business workflow.

15. Recommended Final Direction

Current: “Add chat and voice to your application.”

Recommended: “Add realtime voice, video, chat and AI intelligence to your application through one developer-friendly SDK.”

Long-term vision: “The realtime communication and AI layer that applications can embed instead of building their own media, communication and intelligence infrastructure.”

Strategic path: RTC Engine → Developer Platform → Communication Intelligence → Vertical Workflows → AI Agents

This preserves the engineering work already present in the uploaded specification while creating a stronger reason for customers to choose the product.

Source basis: the uploaded RTC Platform specification. Strategic additions in this document are recommendations/inferences from the review and are not claims that those features are already implemented.